Содержание

Введение 2
1. Проектирование кластера 4
1.1. Технологический стек 4
1.2. Обоснование выбора технологий 5
1.3. Архитектурная схема кластера 6
1.4. Сетевые зоны и роли узлов 8
1.5. Схемы репликации и анализ отказоустойчивости 10
1.6. Размещение сервисов и хранение данных 11
2. Практическая реализация 12
2.1. Описание разработанных сервисов и их функциональных ролей 13
2.2. Подготовка среды и конфигурация оркестратора 14
2.3. Реализация системы централизованного сбора журналов 15
2.4. Развертывание сервера регистрации сервисов 16
2.5. Настройка политик обновления и самовосстановления 17
3. Демонстрация работы 18
3.1. Проверка отказоустойчивости и механизмов самовосстановления 18
3.2. Проверка обновления сервисов в кластере 20
3.3. Демонстрация сбора телеметрии с сервисом визуализации 21
3.4. Демонстрация работы сервера регистрации 24
Заключение 25
Список источников 27
ПРИЛОЖЕНИЕ А – Конфигурация стека 28
ПРИЛОЖЕНИЕ Б – Сервисы 37

Введение

Актуальность работы продиктована широким распространением микросервисных архитектур, которые, в отличие от монолитных, состоят из множества независимых компонентов. Ручное управление такими системами несостоятельно, что делает технологии контейнерной оркестрации необходимым стандартом. Ключевой задачей при этом становится проектирование отказоустойчивых кластеров, обеспечивающих автоматизацию развертывания, масштабирования и восстановления сервисов, что минимизирует простои. Отдельное внимание в работе уделяется обеспечению наблюдаемости распределенных систем через внедрение централизованного сбора телеметрии и механизмов управления конфигурациями, без которых невозможна эффективная эксплуатация современного программного обеспечения.

Целью курсовой работы является изучение методов контейнерной оркестрации и формирование практических навыков проектирования отказоустойчивого кластера на основе Docker Swarm [1], предназначенного для развертывания и устойчивого функционирования микросервисов с поддержкой централизованного мониторинга и управления конфигурациями.

Задание (вариант №9): создать вычислительный кластер из 3 узлов, где 1 мастер и 2 воркера, являющихся отдельными физическими серверами (или виртуальными машинами), в котором:

- мастер узел имеет политику отказоустойчивости "автовостановление через N попыток"

- первый узел с воркерами имеет политику отказоустойчивости для сервисов: "репликация с нечетным количеством копий"

- второй узел с воркерами имеет политику отказоустойчивости для сервисов: "автовостановление через N попыток"

Стратегия обновления сервисов в кластере должна соответствовать политике:

- допускается простой сервиса

- без смешивания версий

- с откатом в случае сбоя при обновлении

Функциональные требования к сервисам:

На первом узле:

- каждый сервис имеет свой отдельный слой данных, к которому он обращается в виде: тома

На втором узле:

- все сервисы имеют единый слой данных, к которому они обращаются в виде: базы данных

Инфраструктура кластера должна обеспечивать функции (задания на оценки 4 и 5):

- на 4: Централизованный сбор телеметрии с сервисом визуализации для просмотра

- на 5: В кластере должен присутствовать: "Сервер конфигураций (сервисы должны получать конфигурации для запуска только с него)" и взаимодействовать с кластером согласно схеме. На Рис. 1 представлена схема кластера.

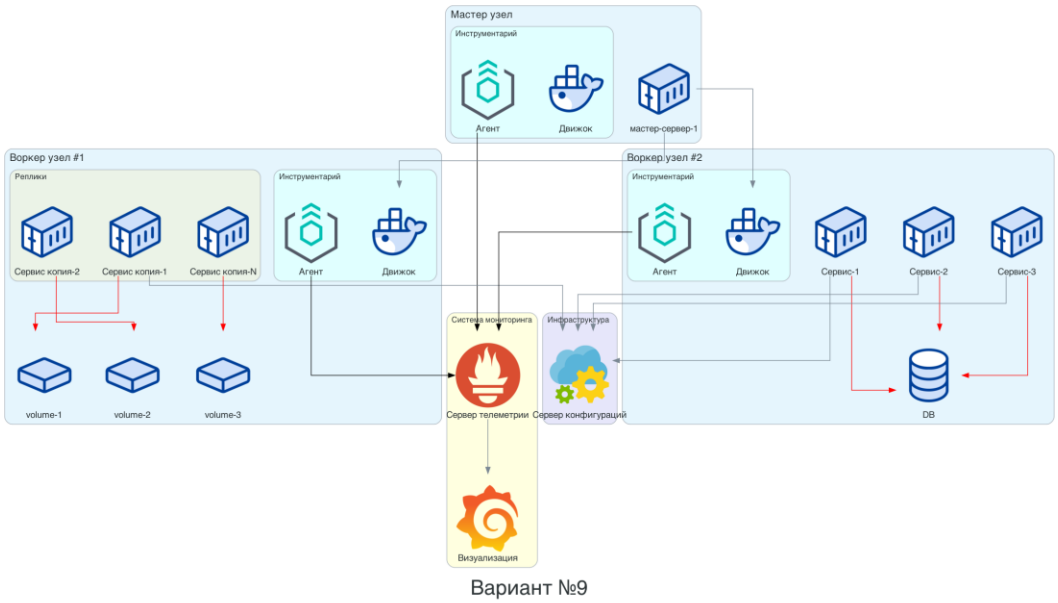


Рис. 1 – Схема кластера

1. Проектирование кластера

В данном разделе рассматривается процесс формирования архитектурных решений, необходимых для создания отказоустойчивой среды выполнения контейнеризированных приложений. Проектирование включает в себя выбор технологического стека, обоснование структуры вычислительных узлов и планирование сетевого взаимодействия. Особое внимание уделяется механизмам обеспечения доступности данных, распределению ролей внутри кластера Docker Swarm для обеспечения кворума и стабильности управляющего контура, а также внедрению системы централизованного мониторинга и управления конфигурациями.

1.1. Технологический стек

Для реализации поставленной задачи выбран стек технологий, обеспечивающий полный цикл управления контейнеризированными приложениями, мониторинг метрик и централизованное хранение конфигураций:

Среда виртуализации: Oracle VM VirtualBox [2]. Позволяет создать изолированную сеть из трёх виртуальных машин на базе Ubuntu Server [3], имитируя распределённую инфраструктуру.

Контейнеризация: Docker Engine [4]. Обеспечивает упаковку приложений и их зависимостей в изолированные контейнеры, гарантируя переносимость и воспроизводимость среды выполнения.

Оркестрация: Docker Swarm [1]. Нативный инструмент Docker для объединения узлов в кластер, управления репликацией сервисов, балансировки нагрузки и реализации политик отказоустойчивости.

Сервер конфигураций: HashiCorp Consul [5]. Используется в режиме Key/Value store для централизованного хранения и распространения конфигурационных параметров сервисов. Каждый сервис при запуске обращается к Consul для получения необходимых настроек.

Система мониторинга: Prometheus [6] + Grafana [7]. Prometheus обеспечивает сбор и хранение метрик, Grafana предоставляет интерфейс для визуализации и анализа данных телеметрии.

Сбор метрик контейнеров: cAdvisor [8]. Агент, работающий в режиме global на всех узлах кластера, собирает метрики использования ресурсов (CPU, память, сеть, файловая система) для каждого контейнера.

Сбор системных метрик: Node Exporter [9]. Предоставляет метрики хост-системы (загрузка CPU, использование памяти, дисковое пространство, сетевая активность) для интеграции с Prometheus.

Мониторинг базы данных: Redis Exporter [10]. Экспортирует метрики Redis (подключения, команды, память) в формате Prometheus.

Управление кластером: Portainer [11]. Веб-интерфейс для визуального управления сервисами, томами, сетями и просмотра статистики в реальном времени.

База данных: Redis [12]. Используется на втором воркер-узле как единое хранилище данных для группы сервисов.

Полную конфигурацию стека, а также содержимое конфигурационных файлов для Prometheus и Grafana см. в Приложении А.

1.2. Обоснование выбора технологий

Выбор Docker Swarm в качестве основного оркестратора обусловлен его эффективностью при работе в условиях ограниченных ресурсов виртуальных машин. В отличие от Kubernetes, Swarm требует значительно меньше оперативной памяти для функционирования управляющего контура (Control Plane) и обладает более низким порогом входа при настройке сетевого взаимодействия (Mesh Routing). Это позволяет сосредоточиться на логике отказоустойчивости и политиках развертывания, не усложняя архитектуру избыточными абстракциями. Нативная поддержка репликации сервисов, политик перезапуска и стратегий обновления делает Swarm оптимальным выбором для реализации требований задания.

Выбор HashiCorp Consul для реализации сервера конфигураций продиктован требованиями задания. Consul предоставляет надёжное хранилище ключ-значение (Key/Value store) с поддержкой HTTP API, что позволяет сервисам динамически получать конфигурационные параметры при запуске. В отличие от статических файлов конфигурации, такой подход обеспечивает централизованное управление настройками: изменение параметра в Consul автоматически применяется всеми зависимыми сервисами при следующем запросе. Веб-интерфейс Consul упрощает аудит и отладку конфигураций, а встроенные механизмы согласованности данных гарантируют целостность настроек в распределённой среде.

Применение стека Prometheus + Grafana для сбора и визуализации телеметрии обосновано архитектурным подходом к мониторингу метрик. Prometheus использует модель pull-based сбора данных, что обеспечивает надёжность доставки метрик даже при временной недоступности сервера. Графики и алерты в Grafana позволяют оперативно выявлять аномалии в работе кластера. В отличие от систем на базе Elasticsearch, ориентированных на полнотекстовый поиск по логам, Prometheus оптимизирован для хранения и агрегации числовых метрик, что снижает нагрузку на дисковую подсистему и ускоряет построение графиков в реальном времени.

1.3. Архитектурная схема кластера

Проектируемый кластер состоит из трёх узлов, связанных в общую виртуальную сеть на базе overlay-драйвера Docker:

Master-узел (KR-MasterT) выполняет функции управления кластером и размещения центральных компонентов инфраструктуры. На нём развёрнуты: сервер конфигураций HashiCorp Consul для централизованного хранения настроек всех сервисов, система мониторинга Prometheus для сбора метрик, Grafana для визуализации телеметрии, а также Portainer для визуального управления кластером. Master-узел имеет политику отказоустойчивости «автовостановление через N попыток» для размещённых на нём сервисов.

Worker-узел №1 (KR-Worker1T) предназначен для размещения группы сервисов app1-1, app1-2, app1-3, каждый из которых использует свой отдельный том (volume) для хранения данных. Данный узел реализует политику отказоустойчивости «репликация с нечетным количеством копий» — каждый сервис запускается в трёх репликах, что гарантирует сохранение работоспособности при выходе из строя одной из реплик и позволяет избежать ситуации split-brain при принятии решений о состоянии сервиса.

Worker-узел №2 (KR-Worker2T) используется для размещения сервисов app2-1, app2-2, app2-3, которые имеют единый слой данных в виде общей базы данных Redis. Все три сервиса обращаются к одному экземпляру Redis для получения и сохранения данных, что реализует требование задания о централизованном хранении данных на втором воркер-узле. Данный узел имеет политику отказоустойчивости «автовостановление через N попыток» — при падении сервиса Docker Swarm предпринимает несколько попыток его перезапуска для восстановления работоспособности.

Для обеспечения отказоустойчивости управляющего контура всем трём узлам назначена роль Manager в режиме Docker Swarm. Это формирует кворум из трёх участников, что позволяет кластеру сохранять управляемость при выходе из строя любого одного узла. Такое решение повышает надёжность всей системы, так как при потере одного узла оставшиеся два могут продолжать функционировать и поддерживать консенсус по алгоритму Raft.

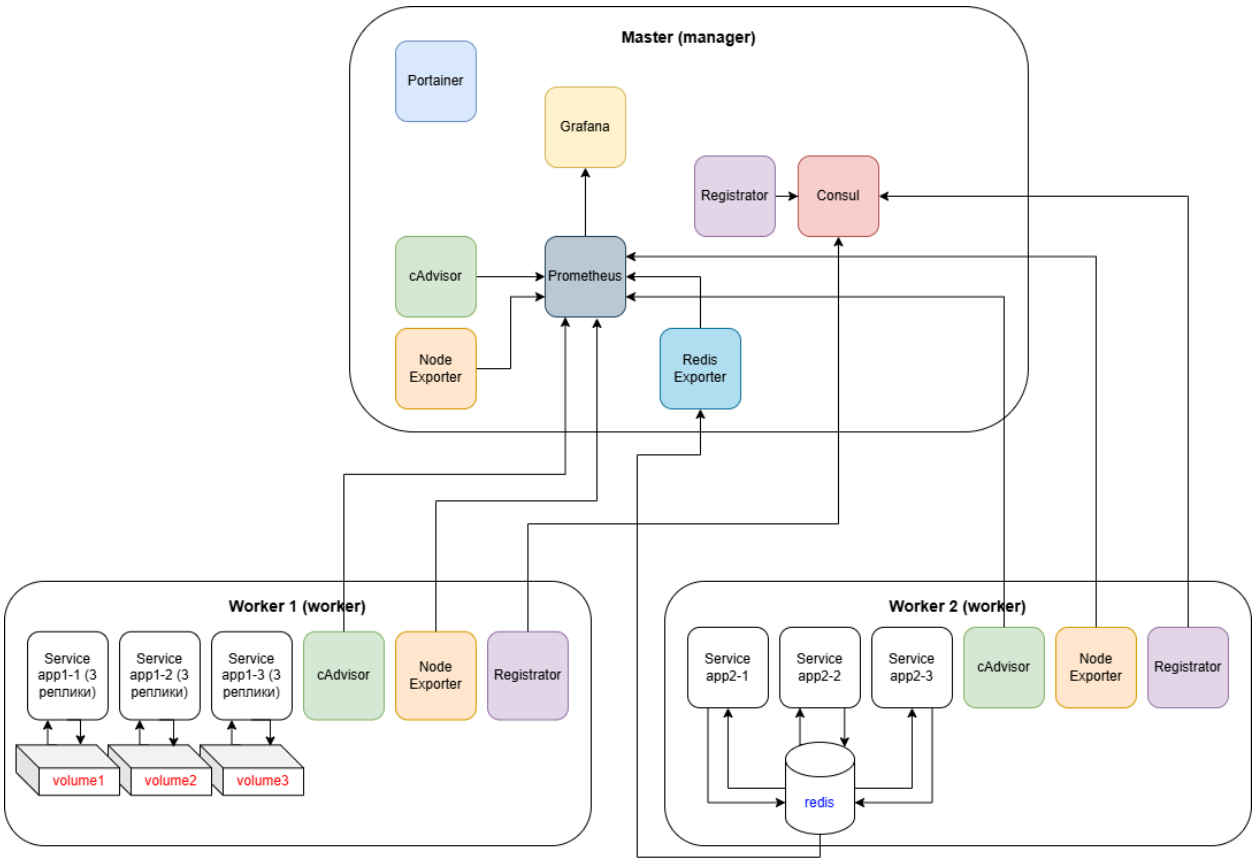


Рис. 1.1 – Архитектурная схема кластера

1.4. Сетевые зоны и роли узлов

Сетевая архитектура разработанного кластера спроектирована с учетом необходимости изоляции внутреннего трафика микросервисов от внешних запросов. Основным инструментом организации межхостового взаимодействия является драйвер Overlay. Эта виртуальная сеть представляет собой распределенный программно-определяемый слой (SDN), который объединяет разрозненные Docker-хосты в единое адресное пространство. Использование протокола инкапсуляции VXLAN позволяет контейнерам, расположенным на физически разных узлах, взаимодействовать по внутренним IP-адресам через шифрованные туннели, что критически важно для безопасности данных в распределенных системах.

Логическая структура кластера строится на четком разграничении ролей вычислительных узлов:

Узлы-менеджеры (Manager / Control Plane): выполняют функции управления и координации. Они формируют управляющий контур,

работающий на базе алгоритма консенсуса Raft. Менеджеры отвечают за хранение и репликацию распределенного состояния кластера (Cluster State), обработку API-запросов и планирование (scheduling) задач. Основная задача менеджеров — непрерывный мониторинг системы: если «фактическое» состояние (например, количество работающих контейнеров) перестает соответствовать «желаемому» (описанному в конфигурации), оркестратор мгновенно инициирует процесс восстановления недостающих реплик. В рамках данной работы роль менеджера выполняет узел KR-MasterT, на котором размещены инфраструктурные компоненты: Consul, Prometheus, Grafana и Portainer.

Рабочие узлы (Worker / Data Plane): предназначены исключительно для выполнения полезной нагрузки. На них разворачиваются инстансы микросервисов, базы данных и агенты сбора метрик. В рамках данной работы два рабочих узла имеют различные роли:

KR-Worker1T: размещает сервисы app1-1, app1-2, app1-3 с индивидуальными томами хранения данных. Каждый сервис имеет собственный volume, что обеспечивает изоляцию данных и независимое масштабирование.

KR-Worker2T: размещает сервисы app2-1, app2-2, app2-3, которые используют единую базу данных Redis. Все сервисы обращаются к общему экземпляру Redis для хранения и получения данных.

Для обеспечения доступности сервисов из внешней сети используется механизм Ingress Routing Mesh. Эта технология реализует встроенную балансировку нагрузки на уровне ядра. Суть механизма заключается в том, что каждый узел кластера прослушивает опубликованные порты сервисов. При поступлении внешнего запроса на любой из узлов (даже если на нем физически не запущен нужный контейнер), Routing Mesh автоматически перенаправляет трафик через внутреннюю сеть на свободный экземпляр сервиса. Это позволяет абстрагировать конечного пользователя от конкретной

IP-адресации узлов и гарантирует доступность системы при частичной деградации инфраструктуры.

1.5. Схемы репликации и анализ отказоустойчивости

Согласно заданию, в кластере реализованы различные политики обеспечения надежности:

Узел Master и управляющий контур: реализована политика автовостановления через N попыток (restart_policy: on-failure, max_attempts: 3). При сбое инфраструктурного сервиса (Consul, Prometheus, Grafana) Docker Swarm предпринимает до трёх попыток его перезапуска с интервалом 5 секунд. Данная политика выбрана в связи с тем, что на мастер-узле размещены уникальные инфраструктурные компоненты, репликация которых не требуется — достаточно обеспечить их автоматическое восстановление при отказе.

Узел Worker-1 (Репликация приложений): для сервисов app1-1, app1-2 и app1-3 установлена репликация в количестве 3 экземпляров. Использование нечетного числа реплик позволяет избежать ситуации split-brain при принятии решений о состоянии группы реплик. Каждый сервис имеет собственный отдельный том (volume) для хранения данных, что обеспечивает изоляцию данных между сервисами. Стратегия обновления stop-first с parallelism: 1 и failure_action: rollback позволяет обновлять приложение последовательно, по одному контейнеру, с возможностью отката в случае сбоя. Допускается кратковременный простой сервиса во время обновления.

Узел Worker-2 (Автовосстановление): для сервисов app2-1, app2-2, app2-3 задана политика restart_policy (condition: on-failure, max_attempts: 3, delay: 5s), обеспечивающая автоматический перезапуск при сбоях. Все три сервиса обращаются к единой базе данных Redis, что реализует требование задания о централизованном хранении данных на втором воркер-узле. Стратегия обновления аналогична Worker-1: stop-first с parallelism: 1 и failure_action: rollback, что гарантирует отсутствие смешивания версий и возможность отката при неудачном обновлении.

1.6. Размещение сервисов и хранение данных

Эффективное распределение нагрузки и изоляция вычислительных ресурсов в кластере реализованы через механизмы Placement Constraints (ограничения размещения). Этот инструментарий позволяет гибко управлять планировщиком Docker Swarm, гарантируя, что конкретные контейнеры будут развернуты только на тех узлах, которые соответствуют заданным критериям (меткам или ролям). В данной работе такой подход применен для имитации разделения сред выполнения и оптимизации использования ресурсов:

Изоляция прикладных сервисов: Для обеспечения строгого разделения нагрузок согласно техническому заданию, сервисы первой группы (app1-1, app1-2, app1-3) жестко привязаны к узлу KR-Worker1T через ограничение node.hostname == KR-Worker1T. Аналогично, сервисы второй группы (app2-1, app2-2, app2-3) закреплены за узлом KR-Worker2T через ограничение node.hostname == KR-Worker2T. Это реализовано через инструкции в блоке deploy конфигурационного файла stack.yml, что предотвращает нецелевое использование ресурсов соседних узлов и гарантирует соблюдение требований задания.

Размещение инфраструктурного слоя: Компоненты системы мониторинга и управления конфигурациями (Consul, Prometheus, Grafana, Portainer) закреплены за узлом с ролью manager (KR-MasterT) через ограничение node.role == manager. Это обусловлено необходимостью обеспечения стабильного доступа к API управления кластером и высокой доступности критически важных сервисов управления, которые должны функционировать независимо от нагрузки на воркеры.

В части организации работы с данными в проекте реализовано два фундаментально разных подхода, выбор которых зависит от характера обрабатываемой информации и требований к её жизненному циклу:

1. Постоянное хранение (Worker-1): Для сервисов группы app1-x, работающих на узле KR-Worker1T, реализован механизм именованных томов (Docker Volumes). Каждый сервис имеет свой

отдельный том: app1-1 использует vol1-1, app1-2 использует vol1-2, app1-3 использует vol1-3. Данные монтируются в директорию /app/data внутри контейнера. Использование именованных томов позволяет абстрагировать управление данными от файловой системы хоста, обеспечивая сохранность информации при обновлении образов, перезагрузке контейнеров или аварийном завершении процессов. Это соответствует требованию задания о том, что «каждый сервис имеет свой отдельный слой данных, к которому он обращается в виде: тома».

2. Централизованное хранение (Worker-2): Для сервисов группы app2-x, функционирующих на узле KR-Worker2T, используется единая база данных Redis. Все три сервиса (app2-1, app2-2, app2-3) обращаются к одному экземпляру Redis через переменные окружения DB_HOST=redis и DB_PORT=6379. Внешние тома для этих сервисов не используются, так как данные хранятся централизованно в Redis. Такой подход реализует требование задания о том, что «все сервисы имеют единый слой данных, к которому они обращаются в виде: базы данных». Redis развернут на том же узле KR-Worker2T с политикой автовостановления при сбоях.

2. Практическая реализация

Второй раздел посвящен техническому описанию процесса развертывания и настройки компонентов системы в соответствии со спроектированной схемой. В нем приводится описание конфигурационных файлов оркестратора, а также спецификации развернутых микросервисов. Рассматриваются этапы интеграции инструментов централизованного сбора телеметрии и системы управления конфигурациями, что позволяет сформировать готовую к эксплуатации инфраструктуру на базе виртуализированных ресурсов.

2.1. Описание разработанных сервисов и их функциональных ролей

В основе прикладного уровня кластера лежат микросервисы, упакованные в Docker-контейнеры. Архитектура разделена на две функциональные группы, каждая из которых решает специфические задачи в рамках учебного проекта.

Первая группа включает сервисы app1-1, app1-2 и app1-3, использующие пользовательский образ zabava27/my-service1:v1, загруженный на Docker Hub. Эти компоненты размещаются на узле KR-Worker1T и имитируют типичную веб-нагрузку. Для демонстрации работы с персистентными данными каждый сервис этой группы имеет собственный именованный том Docker: app1-1 использует vol1-1, app1-2 использует vol1-2, app1-3 использует vol1-3. Данные монтируются в директорию /app/data внутри контейнера. Это позволяет приложению сохранять результаты своей работы на файловой системе хоста. Такая конфигурация необходима для проверки корректности политики репликации с нечетным количеством копий: при перезапуске контейнера после сбоя данные в томе остаются доступными для нового экземпляра процесса, обеспечивая непрерывность бизнес-логики. Для идентификации экземпляров в системе мониторинга используется переменная окружения NODE_NAME.

Вторая группа сервисов (app2-1, app2-2, app2-3) базируется на образе zabava27/my-service2:v1 и размещается на узле KR-Worker2T. В отличие от первой группы, данные сервисы ориентированы на работу с единым централизованным хранилищем данных. Для этого в конфигурации стека предусмотрен общий экземпляр базы данных Redis, к которому обращаются все три сервиса через переменные окружения: DB_HOST=redis и DB_PORT=6379. Взаимодействие настроено через встроенный DNS-сервер Docker Swarm для разрешения имен сервисов. Внешние тома для этих сервисов не используются, так как данные хранятся централизованно в Redis. Такой подход реализует требование задания о том, что «все сервисы имеют единый слой данных, к которому они обращаются в виде: базы данных». Для

идентификации экземпляров в системе мониторинга используется переменная NODE_NAME. Полный код сервисов см. в Приложении Б.

2.2. Подготовка среды и конфигурация оркестратора

Процесс развертывания начался с создания инфраструктурного слоя на базе трёх виртуальных машин под управлением Ubuntu Server 22.04 LTS. Каждая виртуальная машина была настроена вручную с выделением 2 ГБ оперативной памяти и 2 ядер процессора, что обеспечивает достаточные ресурсы для работы контейнеризированных сервисов. Сетевые интерфейсы были сконфигурированы в режиме Host-Only, что обеспечило создание изолированной локальной сети между виртуальными машинами с фиксированными IP-адресами, необходимой для стабильной работы протокола VXLAN в Docker Swarm Overlay-сети.

После подготовки виртуальных машин была проведена инициализация кластера. На узле KR-MasterT была выполнена команда docker swarm init, сформировавшая управляющий контур. Узлы KR-Worker1T и KR-Worker2T были присоединены к кластеру в качестве рабочих нод с помощью команды docker swarm join, используя токен, сгенерированный на мастер-узле. В данной работе реализована схема с одним управляющим узлом, что соответствует требованию задания о политике «автовостановление через N попыток» для мастер-узла. При сбое инфраструктурных сервисов на мастере Docker Swarm предпринимает автоматические попытки их восстановления без необходимости поддержания кворума из нескольких менеджеров.

Для визуализации состояния кластера и упрощения оперативного управления контейнерами в среду был интегрирован графический интерфейс Portainer CE [11]. Развертывание Portainer осуществлялось на узле-менеджере с пробросом Docker Socket, что позволило получить полный контроль над ресурсами Swarm-кластера через веб-интерфейс. Использование Portainer позволило не только отслеживать распределение реплик по узлам в режиме реального времени, но и оперативно анализировать состояние стеков, томов и сетевых интерфейсов, не прибегая к использованию командной строки на

каждом отдельном воркере. Это существенно повысило скорость диагностики системы в ходе проведения тестов отказоустойчивости и имитации сбоев сервисов.

Настройка политик размещения: После инициализации кластера была выполнена конфигурация политик размещения сервисов через механизм placement constraints в файле stack.yml. Для сервисов первой группы (app1-1, app1-2, app1-3) задано ограничение node.hostname == KR-Worker1T, гарантирующее их запуск исключительно на первом воркер-узле. Для сервисов второй группы (app2-1, app2-2, app2-3) задано ограничение node.hostname == KR-Worker2T. Инфраструктурные сервисы (Consul, Prometheus, Grafana) закреплены за мастер-узлом через ограничение node.role == manager.

Интеграция системы мониторинга: Для реализации требования задания о «централизованном сборе телеметрии с сервисом визуализации» в кластер были добавлены компоненты Prometheus и Grafana. Prometheus настроен на сбор метрик со всех узлов через агенты cAdvisor и Node Exporter, работающие в режиме global. Grafana подключена к Prometheus как источник данных и предоставляет дашборды для визуального анализа метрик производительности кластера.

Настройка сервера конфигураций: Для реализации требования задания о «сервере конфигураций» в кластер был добавлен HashiCorp Consul в режиме Key/Value store. Все прикладные сервисы при запуске получают конфигурационные параметры через HTTP API Consul, используя переменные окружения CONSUL_HTTP_ADDR и CONFIG_PATH. Это обеспечивает централизованное управление настройками: изменение конфигурации в Consul автоматически применяется сервисами при следующем запросе, без необходимости пересборки образов или перезапуска стека.

2.3. Реализация системы централизованного сбора журналов

Для обеспечения наблюдаемости за распределенной системой был внедрен стек мониторинга Prometheus, cAdvisor, Node-Exporter, Grafana. В

условиях микросервисной архитектуры прямой просмотр метрик через стандартные инструменты Docker становится неэффективным, поэтому была реализована схема агрегации данных телеметрии в едином хранилище.

Центральным узлом системы является сервис Prometheus, развернутый на мастере. Для его работы используется внешний конфигурационный файл prometheus.yml, подключенный через механизм Docker Configs. Это позволяет изменять параметры сбора данных (интервалы сканирования, цели сбора, правила агрегации) без пересборки образа. Для хранения временных рядов метрик выделен постоянный том prometheus-.

Сбор метрик контейнеров осуществляется агентом cAdvisor, который развернут в режиме deploy: mode: global. Данная настройка гарантирует, что на каждом узле кластера (KR-MasterT, KR-Worker1T, KR-Worker2T) автоматически запустится экземпляр cAdvisor. Агенты монтируют системные директории (/var/run, /sys, /var/lib/docker) в режиме чтения, собирают метрики использования ресурсов всеми контейнерами в реальном времени (CPU, память, сеть, файловая система) и предоставляют их для сбора Prometheus по протоколу HTTP.

Сбор системных метрик хоста осуществляется агентом Node Exporter, также развернутым в режиме global. Node Exporter предоставляет метрики самой хост-системы: загрузку процессора, использование оперативной памяти, дисковое пространство, сетевую активность, температуру компонентов. Эти метрики собираются Prometheus вместе с метриками контейнеров.

Сбор метрик базы данных осуществляется через Redis Exporter, развернутый на мастер-узле. Этот компонент экспортирует метрики Redis (количество подключений, выполненные команды, использование памяти, статистика ключей) в формате Prometheus.

2.4. Развертывание сервера регистрации сервисов

В соответствии с требованиями задания, в кластер интегрирован сервер конфигураций HashiCorp Consul, обеспечивающий функционал

централизованного управления настройками сервисов. В динамических средах, таких как Docker Swarm, где конфигурации могут часто изменяться, наличие единого источника настроек является критически важным для обеспечения согласованности работы компонентов.

Сервис Consul развернут на управляющем узле KR-MasterT в режиме agent -dev с флагом -client=0.0.0.0, что открывает доступ к HTTP API и веб-интерфейсу для внешних запросов. В отличие от режима полноценного кластера Consul, режим -dev оптимален для учебного проекта: он не требует настройки bootstrap-кворума и хранит данные в оперативной памяти, что упрощает развертывание без потери функциональности Key/Value store.

2.5. Настройка политик обновления и самовосстановления

Финальным этапом практической реализации стала тонкая настройка жизненного цикла сервисов через секцию deploy в YAML-файле стека. Политики были подобраны таким образом, чтобы удовлетворить требования задания: допускается простой сервиса, без смешивания версий, с откатом в случае сбоя при обновлении.

Для всех прикладных сервисов установлена стратегия stop-first. Это означает, что при обновлении образа Docker сначала останавливает старый контейнер, дождется его полного завершения и только потом запускает новый. Это гарантирует отсутствие конфликтов за порты и исключает ситуацию, когда две разные версии приложения одновременно обрабатывают запросы. Данная стратегия соответствует требованию задания о «допускается простой сервиса» и «без смешивания версий».

Параметр parallelism: 1 обеспечивает последовательное обновление реплик — по одному контейнеру за раз. Для сервисов первой группы (app1-1, app1-2, app1-3) установлено значение replicas: 3, что соответствует требованию о «репликации с нечетным количеством копий». При обновлении в любой момент времени минимум две реплики остаются в сети, обеспечивая доступность сервиса.

В случае возникновения ошибки при старте новой версии (например, из-за некорректного образа или ошибки конфигурации), параметр failure_action: rollback немедленно инициирует откат к предыдущей стабильной версии сервиса. Это соответствует требованию задания о «с откатом в случае сбоя при обновлении». Дополнительно параметр delay: 5s устанавливает паузу между обновлениями реплик для контроля стабильности запуска.

Для сервисов второй группы (app2-1, app2-2, app2-3) и инфраструктурных сервисов мастер-узла (Consul, Prometheus, Grafana) реализован механизм самовосстановления через политику restart_policy с параметрами condition: on-failure и max_attempts: 3. Параметр delay: 5s устанавливает интервал между попытками перезапуска, предотвращая чрезмерную нагрузку на систему при циклических сбоях.

Параметр window со значением 120s (для мастер-сервисов) позволяет Docker Swarm контролировать стабильность запуска в течение двух минут: если сервис не смог запуститься успешно в течение этого окна после трёх попыток, он помечается как failed и не перезапускается автоматически, что упрощает диагностику критических сбоев.

3. Демонстрация работы

Заключительный раздел содержит результаты функционального тестирования развернутого кластера. Основной целью данных испытаний является проверка работоспособности механизмов самовосстановления сервисов, подтверждение возможности обновления приложений с откатом в случае сбоя, а также визуализация процессов получения конфигураций из централизованного сервера Consul. Представленные ниже сценарии и графические материалы подтверждают соответствие реализованного решения требованиям задания.

3.1. Проверка отказоустойчивости и механизмов самовосстановления

Начальный этап комплексных испытаний развернутого кластера был сфокусирован на проведении верификации механизмов обеспечения высокого уровня доступности прикладных сервисов и оценке эффективности процедур

автоматического самовосстановления контейнеризированных приложений. Данное исследование носило структурированный характер и было разделено на два последовательных и функционально значимых этапа: анализ политик перезапуска сервисов на узле KR-Worker2T и проверка механизма репликации на узле KR-Worker1T.

Тестирование воркер-узла №2 (автовостановление):

Первый этап тестирования был сосредоточен на узле KR-Worker2T, где для сервисов app2-1, app2-2, app2-3 настроена политика restart_policy с условием on-failure. Для демонстрации был принудительно завершен процесс внутри одного из контейнеров сервиса app2-1 командой docker kill. Наблюдение за состоянием сервиса через CLI (docker service ps stack_app2-1) показало, что оркестратор Docker Swarm зафиксировал падение контейнера и, согласно заданному параметру max_attempts: 3, инициировал его немедленный перезапуск. На Рис. 3.1.1 и 3.1.2 показано, что контейнер перешёл в статус Shutdown, однако на его месте сразу появился новый экземпляр в статусе Running, сохранив доступ к единой базе данных Redis.

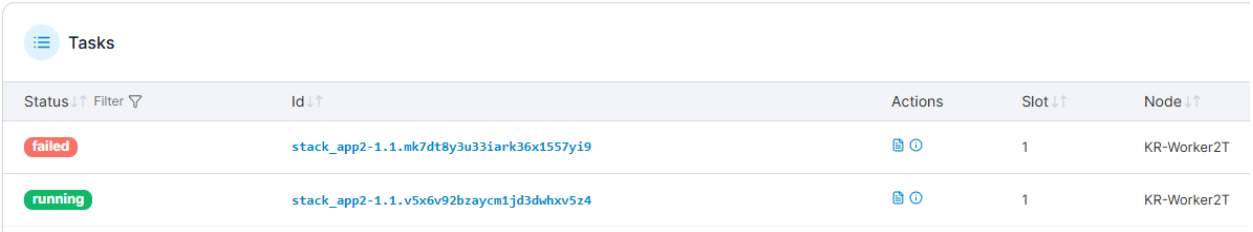


Рис. 3.1 – Демонстрация самовосстановления сервиса app2-1 на Worker-2

Данный результат подтверждает корректность настройки механизма «автовостановление через N попыток» для второго воркер-узла: при возникновении сбоя в работе контейнера система автоматически предпринимает до трёх попыток его перезапуска с интервалом 5 секунд, обеспечивая восстановление работоспособности сервиса без вмешательства администратора.

Тестирование воркер-узла №1 (репликация с нечетным количеством копий):

Второй этап тестирования был посвящён проверке механизма репликации на узле KR-Worker1T. Для сервисов app1-1, app1-2, app1-3 настроено значение replicas: 3, что соответствует требованию задания о «репликации с нечетным количеством копий». Команда docker service ps stack_app1-1 показала, что оркестратор успешно создал и поддерживает ровно три экземпляра каждого сервиса, как указано в конфигурации стека. На Рис. 3.2 видно, что в кластере присутствует ровно три активных реплики сервиса app1-1, три реплики сервиса app1-2 и три реплики сервиса app1-3, каждая из которых имеет доступ к своему отдельному тому (vol1-1, vol1-2 или vol1-3).

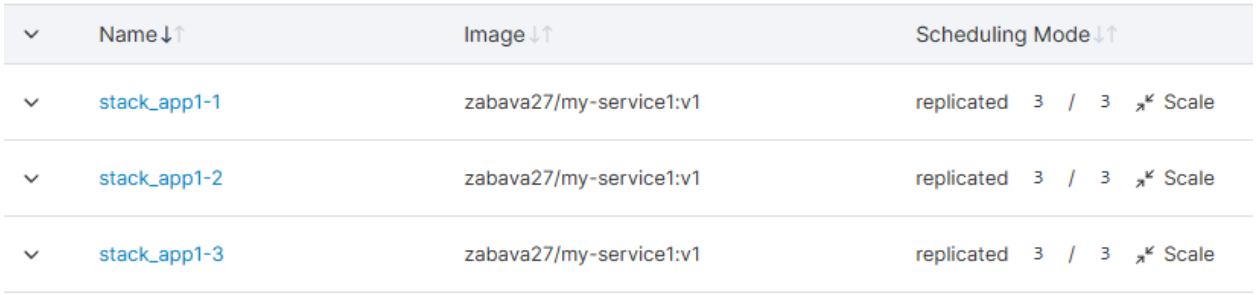


Рис. 3.2 – Демонстрация поддержания репликации сервиса app1-1 на Worker-1

Использование нечетного количества реплик (3) обеспечивает устойчивость группы сервисов к единичным сбоям: при отказе одной реплики две оставшиеся продолжают обрабатывать запросы, сохраняя доступность приложения. Оркестратор Docker Swarm автоматически отслеживает количество активных реплик и при необходимости создаёт новые экземпляры для поддержания заданного значения replicas: 3. При этом каждый сервис изолирован на уровне хранения данных благодаря индивидуальным томам, что предотвращает влияние сбоя одного сервиса на другие и обеспечивает независимое масштабирование.

3.2. Проверка обновления сервисов в кластере

Проведём проверку обновления сервисов в кластере на примере сервиса app1-1. Для этого перейдём по адресу http://KR-Worker1T:8081, на котором работает первый сервис воркера-1. Результат запроса к этой странице представлен на Рис. 3.3.

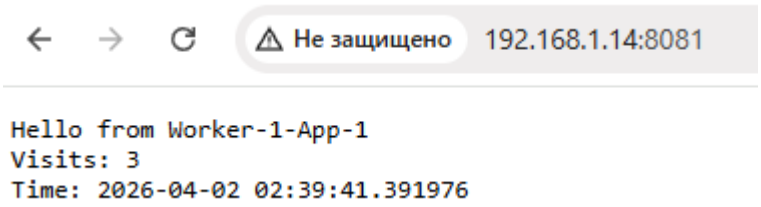


Рис. 3.3 – Страница сервиса app1-1 воркера-1

Затем было проведено обновление образа сервиса с zabava27/my-service1:v1 на zabava27/my-service1:v2. После небольшой задержки, обусловленной стратегией stop-first, версия была успешно обновлена. Результат запроса к странице после обновления сервиса представлен на Рис. 3.4.

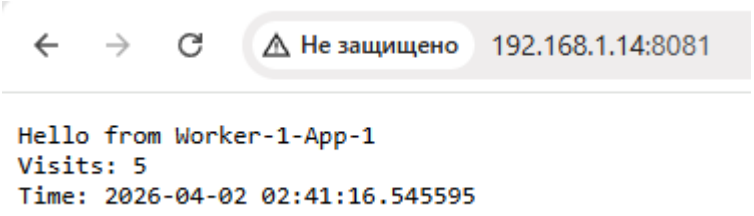


Рис. 3.4 – Результат обновления сервиса

3.3. Демонстрация сбора телеметрии с сервисом визуализации

Для визуализации метрик телеметрии в сервис Grafana был добавлен источник данных Prometheus, после чего создан дашборд для мониторинга использования ресурсов контейнерами. На Рис. 3.5 представлен запрос к Prometheus через интерфейс Grafana с фильтром по имени сервиса: container_label_com_docker_swarm_service_name=~"stack_app1.*". Этот запрос вернул метрики использования CPU и памяти для всех реплик сервиса app1-1, app1-2, app1-3.

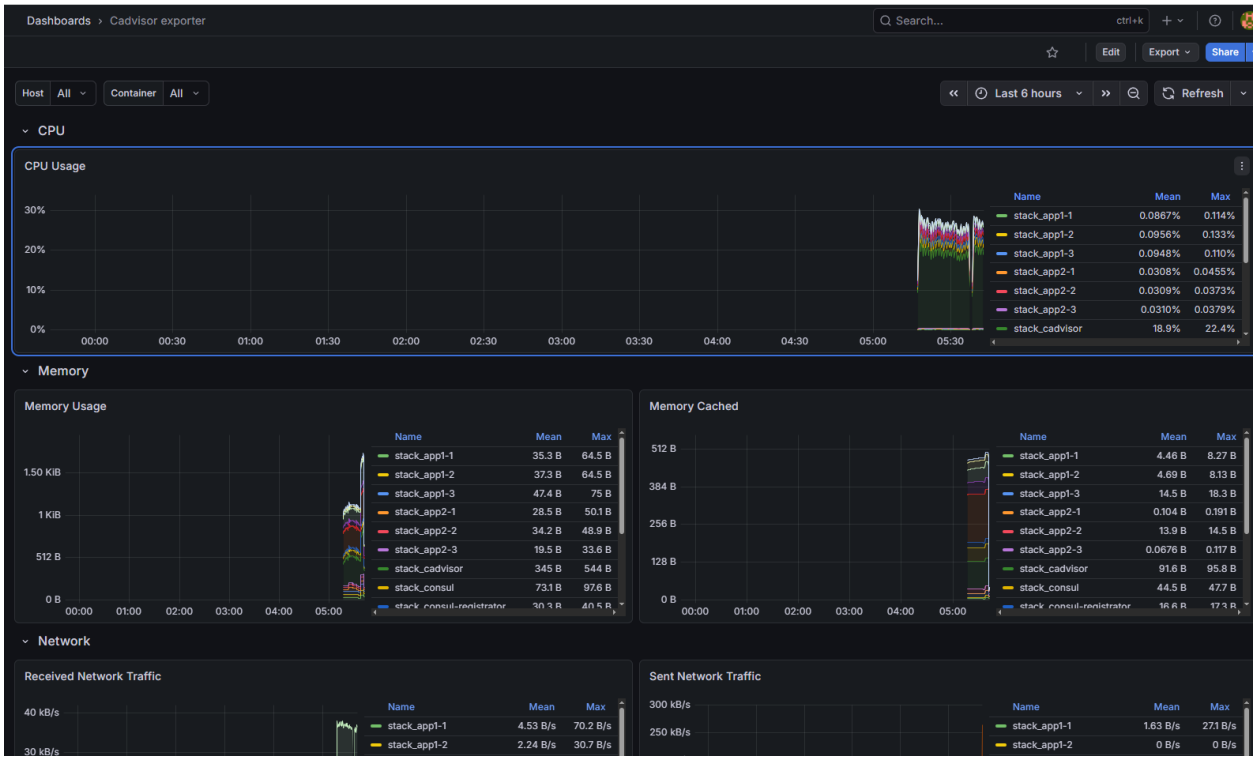


Рис. 3.5 – Просмотр метрик телеметрии из Prometheus в Grafana

Демонстрация ключевых метрик:

- Использование CPU: Запрос sum(rate(container_cpu_usage_seconds_total{container_label_com_docker_swarm_service_name=~"stack_app1.*"}[1m])) by (container_label_com_docker_swarm_service_name) отображает загрузку процессора каждым сервисом в реальном времени.

- Использование памяти: Запрос sum(container_memory_usage_bytes{container_label_com_docker_swarm_service_name=~"stack_app1.*"}) by (container_label_com_docker_swarm_service_name) / 1024 / 1024 показывает потребление оперативной памяти в мегабайтах.

- Сетевой трафик: Запрос sum(rate(container_network_receive_bytes_total{container_label_com_docker_swarm_service_name=~"stack_app1.*"}[1m])) by (container_label_com_docker_swarm_service_name) визуализирует входящий сетевой трафик.

- Системные метрики: Через Node Exporter в Grafana отображаются метрики хост-систем: загрузка ядер процессора, свободное дисковое пространство, активность сети на каждом узле кластера.

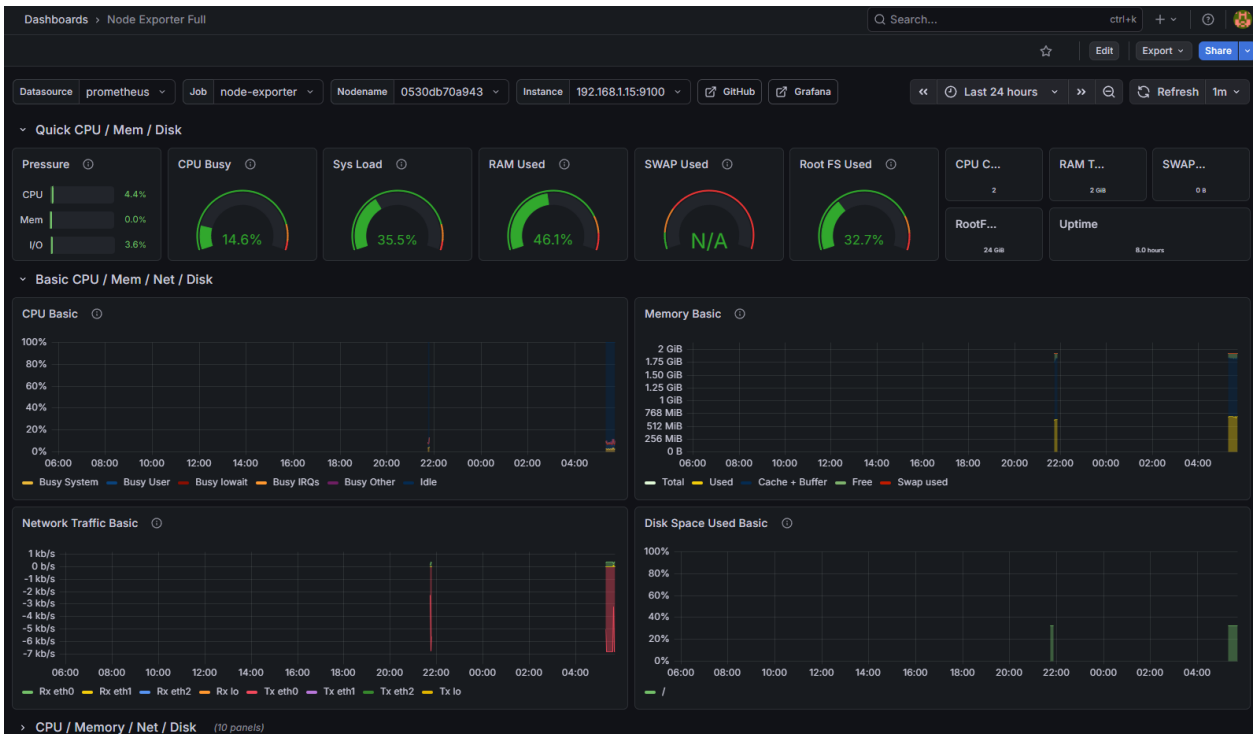


Рис. 3.6 – Дашборд мониторинга кластера в Grafana

Помимо фильтра по имени сервиса в Swarm, имеется возможность фильтровать метрики по имени узла (instance), по меткам контейнеров и по временным интервалам. На Рис. 3.6 представлен сводный дашборд, импортированный из репозитория Grafana (ID: 1860), который отображает состояние всех сервисов кластера, распределение реплик по узлам и ключевые показатели производительности.

Проверка сбора метрик с базы данных:

Через Redis Exporter в Prometheus собираются метрики единой базы данных на воркере-2. Запрос redis_connected_clients показывает количество активных подключений к Redis, а redis_memory_used_bytes — объем используемой памяти. Эти метрики также визуализированы в Grafana, что позволяет отслеживать нагрузку на централизованное хранилище данных.

Таким образом, была продемонстрирована работа системы централизованного сбора телеметрии Prometheus в связке с сервисом визуализации Grafana, что полностью соответствует требованию задания о «централизованном сборе телеметрии с сервисом визуализации для просмотра».

3.4. Демонстрация работы сервера регистрации

Для проверки интеграции системы Service Discovery воспользуемся веб-интерфейсом Consul, который в нашей конфигурации доступен по адресу управляющего узла на порту 8500. В разделе «Services» представлен актуальный реестр всех обнаруженных компонентов системы. Взаимодействие с Docker-контейнерами осуществляется через агент Registrator, который в автоматическом режиме фиксирует запуск новых инстансов и передает их метаданные в реестр.

Как продемонстрировано на Рис. 3.7, сервис my-service представлен в системе 9 активными экземплярами. Данный показатель соответствует расчетным параметрам: на узле worker-1 развернуто три отдельных приложения, каждое из которых, согласно индивидуальному заданию, имеет политику масштабирования в 3 реплики. Тот факт, что все 9 записей появились в Consul мгновенно и без ручного вмешательства, подтверждает корректную настройку механизмов динамической регистрации и их готовность к работе в условиях масштабирования кластера

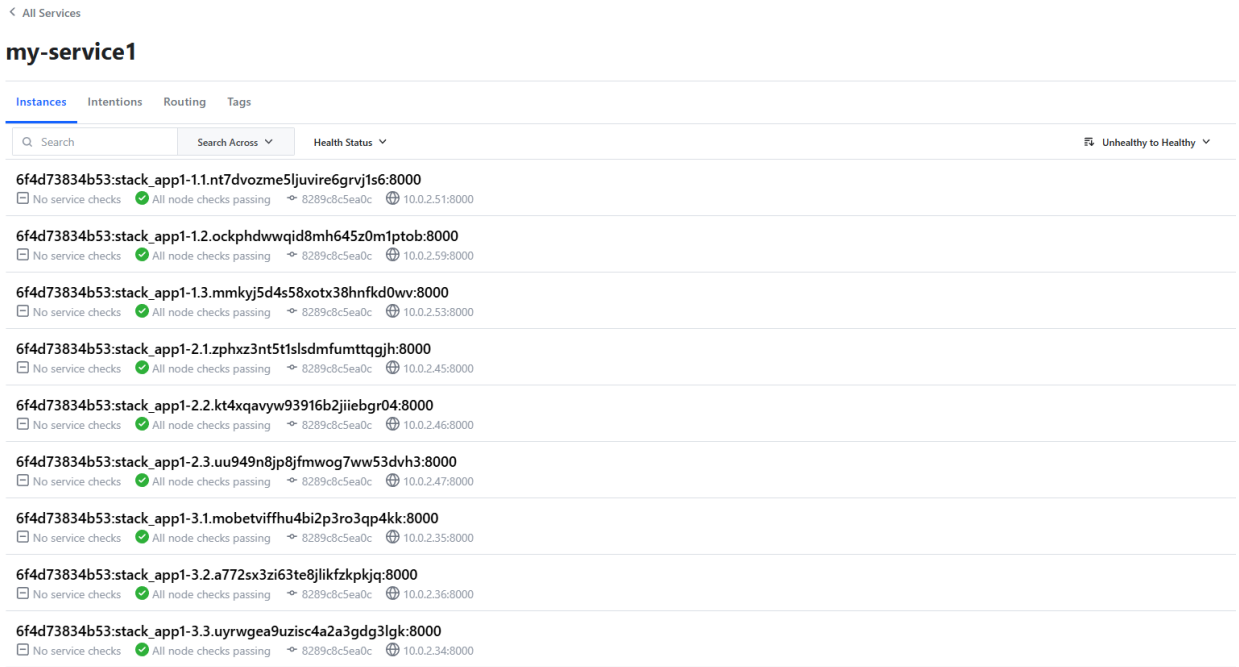


Рис 3.7 – Информация о my-service в Consul

Затем число реплик сервиса app-1-1 было увеличено с 3 до 5, после чего та же самая страница автоматически обновилась, показав актуальные данные,

что представлено на Рис. 3.8. Теперь число экземпляров my-service выросло с 9 до 11, так как в app-1-1 теперь 5 реплик вместо стандартных 3. Таким образом была продемонстрирована работа Consul в связке с агентом регистрации – актуальные данные сервисов кластера были собраны и отображены в пользовательском интерфейсе сервиса

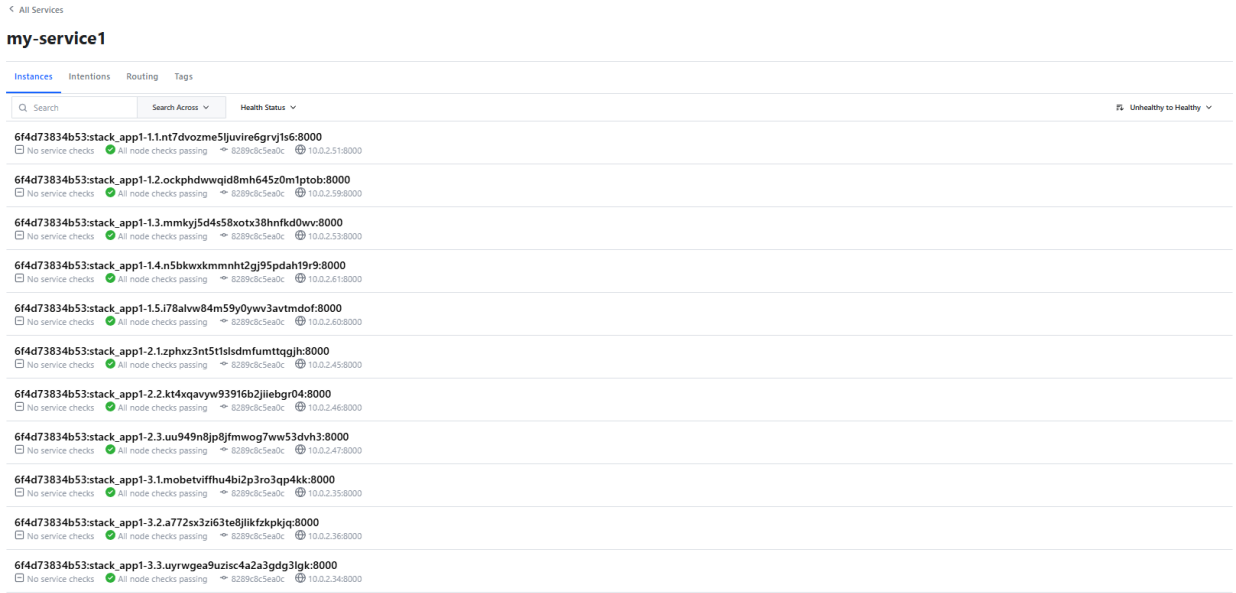


Рис. 3.8 – Данные my-service после масштабирования

Заключение

В ходе курсовой работы был спроектирован и практически реализован отказоустойчивый вычислительный кластер на базе Docker Swarm, соответствующий требованиям индивидуального задания. Работа позволила глубоко изучить принципы контейнерной оркестрации, централизованного управления конфигурациями и мониторинга телеметрии распределённых систем.

Была создана отказоустойчивая инфраструктура из трёх узлов с чётким разделением ролей: один мастер-узел для размещения управляющих компонентов и два воркер-узла для выполнения прикладной нагрузки. Благодаря алгоритму Raft и настроенным политикам перезапуска кластер сохраняет работоспособность при сбое отдельных сервисов. На узлах кластера успешно реализованы и протестированы ключевые стратегии управления: репликация с нечетным количеством копий на первом воркер-узле и

механизмы автовосстановления через N попыток на втором воркер-узле и мастере.

В рамках задания внедрён сервер конфигураций на базе HashiCorp Consul, что обеспечило централизованное получение настроек всеми сервисами при запуске через раздел конфигураций, а также автоматическую регистрацию активных экземпляров в разделе Services. Это позволяет отслеживать состояние всех запущенных сервисов кластера в реальном времени и обеспечивает согласованность конфигураций между компонентами системы. Для обеспечения наблюдаемости развёрнут стек Prometheus, cAdvisor, Node-Exporter, Grafana, который позволил организовать централизованный сбор и визуализацию метрик всего кластера в реальном времени.

В процессе работы были приобретены практические навыки работы с Docker API, Overlay-сетями, механизмами Placement Constraints, политиками развертывания и обновлений, а также ручной настройки виртуальной инфраструктуры на базе Ubuntu Server. Таким образом, цель работы достигнута, а разработанный кластер представляет собой готовую основу для развертывания высокодоступных микросервисных приложений с автоматизированным управлением конфигурациями и наблюдаемостью.

Список источников

1. Docker Swarm Documentation [Электронный ресурс]. – URL: https://docs.docker.com/engine/swarm/ (дата обращения: 02.03.2026).
2. Oracle VM VirtualBox Documentation [Электронный ресурс]. – URL: https://www.virtualbox.org/wiki/Documentation (дата обращения: 02.03.2026).
3. Ubuntu Server Documentation [Электронный ресурс]. – URL: https://ubuntu.com/server/docs (дата обращения: 02.03.2026).
4. Docker Engine Documentation [Электронный ресурс]. – URL: https://docs.docker.com/engine/ (дата обращения: 02.03.2026).
5. HashiCorp Consul Documentation [Электронный ресурс]. – URL: https://developer.hashicorp.com/consul/docs (дата обращения: 04.03.2026).
6. Prometheus Documentation [Электронный ресурс]. – URL: https://prometheus.io/docs/ (дата обращения: 04.03.2026).
7. Grafana Documentation [Электронный ресурс]. – URL: https://grafana.com/docs/grafana/latest/ (дата обращения: 04.03.2026).
8. cAdvisor Documentation [Электронный ресурс]. – URL: https://github.com/google/cadvisor (дата обращения: 04.03.2026).
9. Node Exporter Documentation [Электронный ресурс]. – URL: https://github.com/prometheus/node_exporter (дата обращения: 04.03.2026).
10. Redis Exporter Documentation [Электронный ресурс]. – URL: https://github.com/oliver006/redis_exporter (дата обращения: 05.03.2026).
11. Portainer Documentation [Электронный ресурс]. – URL: https://docs.portainer.io/ (дата обращения: 05.03.2026).
12. Redis Documentation [Электронный ресурс]. – URL: https://redis.io/docs/ (дата обращения: 05.03.2026).

ПРИЛОЖЕНИЕ А – Конфигурация стека

stack.yml:

```
version: '3.8'

services:
  consul:
    image: hashicorp/consul:1.15
    networks:
      - app-network
    command: "agent -dev -client=0.0.0.0"
    ports:
      - "8500:8500"
    deploy:
      placement:
        constraints: [node.role == manager]
      replicas: 1

  app1-1:
    image: zabava27/my-service1:v1
    volumes:
      - vol1-1:/app/data
    environment:
      - NODE_NAME=Worker-1-App-1
      - CONSUL_HTTP_ADDR=consul:8500
      - CONFIG_PATH=config/worker1/app1
    ports:
      - "8081:8000"
    networks:
      - app-network
    deploy:
      replicas: 3
      placement:
        constraints: [node.hostname == KR-Worker1T]
      restart_policy:
        condition: none
      update_config:
        parallelism: 1
        order: stop-first
        failure_action: rollback
        delay: 5s
```

```
  depends_on:
    - consul

app1-2:
  image: zabava27/my-service1:v1
  volumes:
    - vol1-2:/app/data
  environment:
    - NODE_NAME=Worker-1-App-2
    - CONSUL_HTTP_ADDR=consul:8500
    - CONFIG_PATH=config/worker1/app2
  ports:
    - "8082:8000"
  networks:
    - app-network
  deploy:
    replicas: 3
    placement:
      constraints: [node.hostname == KR-Worker1T]
    restart_policy:
      condition: none
    update_config:
      parallelism: 1
      order: stop-first
      failure_action: rollback
      delay: 5s
  depends_on:
    - consul

app1-3:
  image: zabava27/my-service1:v1
  volumes:
    - vol1-3:/app/data
  environment:
    - NODE_NAME=Worker-1-App-3
    - CONSUL_HTTP_ADDR=consul:8500
    - CONFIG_PATH=config/worker1/app3
  ports:
    - "8083:8000"
  networks:
```

```
    - app-network
  deploy:
    replicas: 3
    placement:
      constraints: [node.hostname == KR-Worker1T]
    restart_policy:
      condition: none
    update_config:
      parallelism: 1
      order: stop-first
      failure_action: rollback
      delay: 5s
  depends_on:
    - consul

redis:
  image: redis:alpine
  networks:
    - app-network
  deploy:
    placement:
      constraints: [node.hostname == KR-Worker2T]
    restart_policy:
      condition: on-failure
      max_attempts: 3
      delay: 5s
  depends_on:
    - consul

app2-1:
  image: zabava27/my-service2:v1
  environment:
    - NODE_NAME=Worker-2-App-1
    - CONSUL_HTTP_ADDR=consul:8500
    - CONFIG_PATH=config/worker2/app1
    - DB_HOST=redis
    - DB_PORT=6379
  ports:
    - "8091:8000"
  networks:
```

```
    - app-network
  deploy:
    placement:
      constraints: [node.hostname == KR-Worker2T]
    restart_policy:
      condition: on-failure
      max_attempts: 3
      delay: 5s
    update_config:
      parallelism: 1
      order: stop-first
      failure_action: rollback
      delay: 5s
  depends_on:
    - consul

app2-2:
  image: zabava27/my-service2:v1
  environment:
    - NODE_NAME=Worker-2-App-2
    - CONSUL_HTTP_ADDR=consul:8500
    - CONFIG_PATH=config/worker2/app2
    - DB_HOST=redis
    - DB_PORT=6379
  ports:
    - "8092:8000"
  networks:
    - app-network
  deploy:
    placement:
      constraints: [node.hostname == KR-Worker2T]
    restart_policy:
      condition: on-failure
      max_attempts: 3
      delay: 5s
    update_config:
      parallelism: 1
      order: stop-first
      failure_action: rollback
      delay: 5s
```

```
  depends_on:
    - consul

app2-3:
  image: zabava27/my-service2:v1
  environment:
    - NODE_NAME=Worker-2-App-3
    - CONSUL_HTTP_ADDR=consul:8500
    - CONFIG_PATH=config/worker2/app3
    - DB_HOST=redis
    - DB_PORT=6379
  ports:
    - "8093:8000"
  networks:
    - app-network
  deploy:
    placement:
      constraints: [node.hostname == KR-Worker2T]
    restart_policy:
      condition: on-failure
      max_attempts: 3
      delay: 5s
    update_config:
      parallelism: 1
      order: stop-first
      failure_action: rollback
      delay: 5s
  depends_on:
    - consul

master-service:
  image: alpine:latest
  command: sh -c "while true; do echo 'Master running'; sleep 10; done"
  environment:
    - CONSUL_HTTP_ADDR=consul:8500
    - CONFIG_PATH=config/master/main
  networks:
    - app-network
  deploy:
    placement:
```

```
      constraints: [node.role == manager]
  depends_on:
    - consul

cadvisor:
  image: gcr.io/cadvisor/cadvisor:latest
  ports:
    - "8080:8080"
  volumes:
    - /:/rootfs:ro
    - /var/run:/var/run:ro
    - /sys:/sys:ro
    - /var/lib/docker/:/var/lib/docker:ro
    - /dev/disk/:/dev/disk:ro
  privileged: true
  networks:
    - app-network
  deploy:
    mode: global

prometheus:
  image: prom/prometheus:latest
  ports:
    - "9090:9090"
  volumes:
    - prometheus-data:/prometheus
  configs:
    - source: prometheus_config
      target: /etc/prometheus/prometheus.yml
  command:
    - '--config.file=/etc/prometheus/prometheus.yml'
    - '--storage.tsdb.path=/prometheus'
  networks:
    - app-network
  deploy:
    placement:
      constraints: [node.role == manager]
    replicas: 1

grafana:
```

```
  image: grafana/grafana:latest
  ports:
    - "3000:3000"
  volumes:
    - grafana-data:/var/lib/grafana
  environment:
    - GF_SECURITY_ADMIN_USER=admin
    - GF_SECURITY_ADMIN_PASSWORD=admin123
    - GF_USERS_ALLOW_SIGN_UP=false
    - GF_METRICS_ENABLED=true
    - GF_METRICS_BASIC_COUNTERS_ENABLED=true
    - GF_METRICS_DISABLE_TOTAL_STATS=false
  networks:
    - app-network
  deploy:
    placement:
      constraints: [node.role == manager]
    replicas: 1

node-exporter:
  image: prom/node-exporter:latest
  ports:
    - "9100:9100"
  volumes:
    - /proc:/host/proc:ro
    - /sys:/host/sys:ro
    - /:/rootfs:ro
  command:
    - '--path.procfs=/host/proc'
    - '--path.sysfs=/host/sys'
  networks:
    - app-network
  deploy:
    mode: global

redis-exporter:
  image: oliver006/redis_exporter:latest
  environment:
    - REDIS_ADDR=redis://redis:6379
  networks:
```

```
      - app-network
    deploy:
      placement:
        constraints: [node.role == manager]
      replicas: 1

  consul-registrator:
    image: gliderlabs/registrator:master
    volumes:
      - /var/run/docker.sock:/tmp/docker.sock:ro
    command: -internal=true consul://192.168.1.15:8500
    networks:
      - app-network
    deploy:
      mode: global
    depends_on:
      - consul

networks:
  app-network:
    driver: overlay

volumes:
  vol1-1:
  vol1-2:
  vol1-3:
  prometheus-data:
  grafana-data:

configs:
  prometheus_config:
    external: true
```

prometheus_config.yml:

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'prometheus'
```

```
  static_configs:
    - targets: ['prometheus:9090']

- job_name: 'cadvisor'
  static_configs:
    - targets:
      - '192.168.1.15:8080'
      - '192.168.1.14:8080'
      - '192.168.1.16:8080'

- job_name: 'node-exporter'
  static_configs:
    - targets:
      - '192.168.1.15:9100'
      - '192.168.1.14:9100'
      - '192.168.1.16:9100'

- job_name: 'redis-exporter'
  static_configs:
    - targets: ['redis-exporter:9121']
```

ПРИЛОЖЕНИЕ Б – Сервисы

app1.py:

```
from http.server import HTTPServer, BaseHTTPRequestHandler
import os
from datetime import datetime

NODE_NAME = os.getenv('NODE_NAME', 'Worker-1-App')
DATA_DIR = '/app/data'

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        visits = self.read_visits()
        visits += 1
        self.save_visits(visits)

        message = f'Hello from {NODE_NAME}\nVisits: {visits}\nTime: {datetime.now()}'

        self.send_response(200)
        self.send_header('Content-type', 'text/plain')
        self.end_headers()
        self.wfile.write(message.encode())

    def read_visits(self):
        try:
            with open(f'{DATA_DIR}/visits.txt', 'r') as f:
                return int(f.read().strip())
        except:
            return 0

    def save_visits(self, count):
        os.makedirs(DATA_DIR, exist_ok=True)
        with open(f'{DATA_DIR}/visits.txt', 'w') as f:
            f.write(str(count))

    def log_message(self, format, *args):
```

```
        print(f"[{NODE_NAME}] {args[0]}")

if __name__ == '__main__':
    print(f"Starting server on {NODE_NAME}...")
    HTTPServer(('0.0.0.0', 8000), Handler).serve_forever()
```

app2.py:

```
from http.server import HTTPServer, BaseHTTPRequestHandler
import redis
import os

DB_HOST = os.getenv('DB_HOST', 'localhost')
DB_PORT = int(os.getenv('DB_PORT', 6379))
NODE_NAME = os.getenv('NODE_NAME', 'Worker-2-App')

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        try:
            r = redis.Redis(host=DB_HOST, port=DB_PORT, decode_responses=True)
            visits = r.incr('counter')
            message = f'Hello from {NODE_NAME}\nVisits: {visits}\nDB: {DB_HOST}'
        except Exception as e:
            message = f'Hello from {NODE_NAME}\nDB Error: {e}'

        self.send_response(200)
        self.send_header('Content-type', 'text/plain')
        self.end_headers()
        self.wfile.write(message.encode())

    def log_message(self, format, *args):
        print(f"[{NODE_NAME}] {args[0]}")

if __name__ == '__main__':
    print(f"Starting server on {NODE_NAME}...")
```

```
HTTPServer(('0.0.0.0', 8000), Handler).serve_forever()
```